

16 - Recursion on Non-Numerics



Repaso: Fibonacci y memoization

La función de Fibonacci es el ejemplo clásico de recursión simple pero costosa. Cada llamada genera **dos más**, que a su vez generan dos más... hasta que todo explota en un bosque de llamadas redundantes.

```
1 def fib(x):
2     if x == 0 or x == 1:
3         return 1
4     else:
5         return fib(x-1) + fib(x-2)
```

Llamar `fib(5)` parece trivial, pero implica **15 llamadas**. ¿Por qué? Porque el árbol de ejecución se ramifica exponencialmente.



Memoization: memoria contra el olvido

Memoization (del inglés *memo* = "nota") es una técnica que consiste en **guardar resultados de funciones costosas** en una estructura (como un diccionario), para **no volver a calcularlos** si ya los habíamos visto antes.

Es como un sabio que, al resolver un problema, deja una nota en la pared. Si alguien más se encuentra con el mismo problema, simplemente **lee la nota**.

Con esta técnica, `fib(34)` pasa de hacer **más de 11 millones de llamadas** a apenas **65**. Una auténtica revolución.



Contar combinaciones: el ejemplo del puntaje

Una aplicación más práctica: ¿cuántas formas hay de sumar un puntaje `x` en básquet, si los tiros valen 1, 2 o 3 puntos?

```
1 def score_count(x):
2     if x == 1:
3         return 1
4     elif x == 2:
5         return 2
6     elif x == 3:
```

```
7         return 3
8     else:
9         return score_count(x-1) + score_count(x-2) + score_count(x-3)
```

La magia está en que **cada valor posible de tiro "resta" puntos** y nos deja con un subproblema del mismo tipo. Así se construye un árbol de posibilidades.

Ejemplo:

Para `x = 4`, las formas posibles son:

- 1+1+1+1
- 1+1+2
- 1+2+1
- 2+1+1
- 2+2
- 1+3
- 3+1

⚠ Esta función también **se puede optimizar con memoization** si el `x` es grande.

Listas: la estructura recursiva por naturaleza

Las listas son las reinas de la recursión no numérica. ¿Por qué? Porque son **estructuras fractales**: una lista es una cabeza y una cola. Y esa cola... también es una lista.

+ Sumar elementos (total_recur)

```
1 def total_recur(L):
2     if len(L) == 1:
3         return L[0]
4     else:
5         return L[0] + total_recur(L[1:])
```

Como un peregrino que avanza uno a uno por la senda, esta función suma el primer elemento y confía en que sus sucesores harán lo mismo.

🔍 Buscar un elemento (in_list)

```
1 def in_list(L, e):
2     if L == []:
```

```

3         return False
4     elif L[0] == e:
5         return True
6     else:
7         return in_list(L[1:], e)

```

Evitá el error común de asumir que si no está en la última posición, no está en ningún lado. Primero **mirá al frente**, después al resto.

Aplanando estructuras: `flatten`

✳ *Flatten: aplanar una lista con sublistas*

```

1 def flatten(L):
2     if len(L) == 1:
3         return L[0]
4     else:
5         return L[0] + flatten(L[1:])

```

Solo funciona si las sublistas están a **un solo nivel** de profundidad.

```

1 flatten([[1, 2], [3, 4], [5]]) # → [1, 2, 3, 4, 5]

```

Si tenés sublistas dentro de sublistas, necesitás una versión **profunda** (deep flatten), que no está en esta clase, pero es posible extenderla.

Invirtiendo todo: reversa profunda `deep_rev`

Ahora no queremos solo revertir `[1, [2, [3, 4]], 5]` a `[5, [2, [3, 4]], 1]`, sino **revertir cada lista también internamente**.

```

1 def deep_rev(L):
2     if L == []:
3         return []
4     elif type(L[0]) != list:
5         return deep_rev(L[1:]) + [L[0]]
6     else:
7         return deep_rev(L[1:]) + [deep_rev(L[0])]

```

Este enfoque **invierte recursivamente cada sublista**, como un espejo que refleja no solo el contorno, sino todos los detalles internos.

Palíndromos

La recursión también es útil para muchos problemas que no involucran números. El siguiente código contiene una función, `is_palindrome`, que verifica si una cadena se lee igual hacia adelante y hacia atrás.

Prueba de palíndromo

```
1  def is_palindrome(s):
2      """Asume que s es una str
3      Devuelve True si las letras en s forman un palíndromo; False
4      en caso contrario. Las no-letras y la capitalización son ignoradas."""
5
6      def to_chars(s):
7          s = s.lower()
8          letters = ''
9          for c in s:
10             if c in 'abcdefghijklmnopqrstuvwxyz':
11                 letters = letters + c
12             return letters
13
14         def is_pal(s):
15             if len(s) <= 1:
16                 return True
17             else:
18                 return s[0] == s[-1] and is_pal(s[1:-1])
19
20         return is_pal(to_chars(s))
```

La función `is_palindrome` contiene dos **funciones auxiliares internas**. Esto no debería ser de interés para los clientes de la función, quienes solo deberían preocuparse de que la implementación de `is_palindrome` cumpla con su especificación. Pero tú deberías preocuparte, porque hay cosas que aprender examinando la implementación.

La función auxiliar `to_chars` convierte todas las letras a minúsculas y elimina todas las no-letras. Comienza usando un método incorporado en las cadenas para generar una cadena que es idéntica a `s`, excepto que todas las letras mayúsculas han sido convertidas a minúsculas.

La función auxiliar `is_pal` usa recursión para hacer el trabajo real. Los dos casos base son cadenas de longitud cero o uno. Esto significa que la parte recursiva de la implementación se alcanza solo en cadenas de longitud dos o más. La conjunción en la cláusula `else` se evalúa de izquierda a derecha. El código primero verifica si el primer y último carácter son iguales, y si lo son, continúa verificando si la cadena menos esos dos caracteres es un palíndromo. Que la segunda conjunción no se evalúe a menos que la primera conjunción se evalúe como `True` es semánticamente irrelevante en este ejemplo.

Esta implementación de `is_palindrome` es un ejemplo de un importante principio de resolución de problemas conocido como **dividir y conquistar** (Este principio está relacionado con, pero es ligeramente diferente de, los algoritmos de dividir y conquistar). El principio de esta resolución de problemas es *conquistar un problema difícil dividiéndolo en un conjunto de subproblemas* con las propiedades:

- Los subproblemas son más fáciles de resolver que el problema original.
- Las soluciones de los subproblemas pueden combinarse para resolver el problema original.

Dividir y conquistar es una idea antigua. Julio César practicó lo que los romanos denominaban *divide et impera* (divide y vencerás). Los británicos practicaron brillantemente el control del subcontinente indio, y Benjamin Franklin estaba bien consciente de la experiencia británica al usar esta técnica, lo que le llevó a decir en la firma de la Declaración de Independencia de los Estados Unidos: "Debemos mantenernos todos unidos, o con seguridad nos colgarán por separado."

En este caso, resolvemos el problema dividiendo el problema original en una versión más simple del mismo problema (verificar si una cadena más corta es un palíndromo) y algo simple que sabemos cómo hacer (comparar caracteres individuales), y luego combinamos las soluciones con el operador lógico `and`. El siguiente código puede usarse para visualizar como funciona.

Código para visualizar la prueba de palíndromo

```
1  def is_palindrome(s):
2      """Asume que s es una str
3      Devuelve True si s es un palíndromo; False en caso contrario.
4      Los signos de puntuación, espacios en blanco y capitalización son
       ignorados."""
5
6      def to_chars(s):
7          s = s.lower()
8          letters = ''
9          for c in s:
10             if c in 'abcdefghijklmnopqrstuvwxyz':
11                 letters = letters + c
12         return letters
13
14     def is_pal(s):
15         print(' is_pal called with', s)
16         if len(s) <= 1:
17             print(' About to return True from base case')
18             return True
19         else:
```

```
20         answer = s[0] == s[-1] and is_pal(s[1:-1])
21         print(' About to return', answer, 'for', s)
22         return answer
23
24     return is_pal(to_chars(s))
```

Ejecutando el código

```
1  print('Try dogGod')
2  print(is_palindrome('dogGod'))
3  print('Try doGood')
4  print(is_palindrome('doGood'))
```

Imprime

```
1  Try dogGod
2    is_pal called with doggod
3    is_pal called with oggo
4    is_pal called with gg
5    is_pal called with
6    About to return True from base case
7    About to return True for gg
8    About to return True for oggo
9    About to return True for doggod
10 True
11 Try doGood
12    is_pal called with doggood
13    is_pal called with ogoo
14    is_pal called with go
15    About to return False for go
16    About to return False for ogoo
17    About to return False for dogood
18 False
```

Variables Globales

Si intentaste llamar a `fib` con un número grande, probablemente notaste que tomó mucho tiempo ejecutarse. Supongamos que queremos saber cuántas llamadas recursivas se realizan. Podríamos hacer un análisis cuidadoso del código y calcularlo. Otro enfoque es agregar algo de código para contar el número de llamadas. Una forma de hacer eso usa **variables globales**.

Hasta ahora, todas las funciones que hemos escrito se comunican con su entorno únicamente a través de sus parámetros y valores de retorno. En

su mayor parte, esto es exactamente como debería ser. Típicamente lleva a programas que son relativamente fáciles de leer, probar y depurar. Sin embargo, de vez en cuando, las variables globales resultan útiles.

Usando una variable global

```
1  def fib(x):
2      """Asume x un int >= 0
3      Devuelve el x-ésimo número de Fibonacci"""
4      global num_fib_calls
5      num_fib_calls += 1
6      if x == 0 or x == 1:
7          return 1
8      else:
9          return fib(x-1) + fib(x-2)
10
11 def test_fib(n):
12     for i in range(n+1):
13         global num_fib_calls
14         num_fib_calls = 0
15         print('fib of', i, '=', fib(i))
16         print('fib called', num_fib_calls, 'times.')
```

En cada función, la línea de código `global num_fib_calls` le dice a Python que el nombre `num_fib_calls` debería definirse fuera de la función en la que aparece la línea de código. Si no hubiéramos incluido el código `global num_fib_calls`, el nombre `num_fib_calls` habría sido local a cada una de las funciones `fib` y `test_fib`, porque `num_fib_calls` aparece en el lado izquierdo de una declaración de asignación en tanto `fib` como `test_fib`. Las funciones `fib` y `test_fib` ambas tienen acceso sin restricciones al objeto referenciado por la variable `num_fib_calls`. La función `test_fib` vincula `num_fib_calls` a 0 cada vez que llama a `fib`, y `fib` incrementa el valor de `num_fib_calls` cada vez que se ingresa a `fib`.

La llamada `test_fib(6)` produce la salida

```
1  fib of 0 = 1
2  fib called 1 times.
3  fib of 1 = 1
4  fib called 1 times.
5  fib of 2 = 2
6  fib called 3 times.
7  fib of 3 = 3
8  fib called 5 times.
9  fib of 4 = 5
10 fib called 9 times.
11 fib of 5 = 8
```

```
12  fib called 15 times.
13  fib of 6 = 13
14  fib called 25 times.
```

Introducimos el tema de las variables globales con cierta inquietud. Desde los años 1970, los científicos de la computación portadores de credenciales han arremetido contra ellas, por buena razón. El uso indiscriminado de variables globales puede llevar a muchos problemas. La clave para hacer programas legibles es la localidad. Las personas leen programas una porción a la vez, y mientras menos contexto necesiten para entender cada pieza, mejor. Dado que las variables globales pueden modificarse o leerse en una amplia variedad de lugares, su uso descuidado puede destruir la localidad. Sin embargo, hay unas pocas ocasiones en las que son justo lo que se necesita. El uso más común de las variables globales es probablemente definir una **constante global** que será usada en muchos lugares. Por ejemplo, alguien escribiendo un programa relacionado con la física podría querer definir la velocidad de la luz, C , una vez, y luego usarla en múltiples funciones.

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```
1  def flatten(L):
2      """
3      L: a list
4      Returns a copy of L, which is a flattened version of L
5      """
6      if L == []:
7          return []
8      else:
9          if isinstance(L[0], list):
10             return flatten(L[0]) + flatten(L[1:])
11          else:
12             return [L[0]] + flatten(L[1:])
```